COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/2/26	**DATE SUBMITTED:**	10/2/26
NAME:	SUBALISID, JAPPETH ROYCE B.	**INSTRUCTOR:**	ENGR. JIMLORD M. QUEJADO

Hands-on Activity 10.1 Graphs

1. PROCEDURE OUTPUT

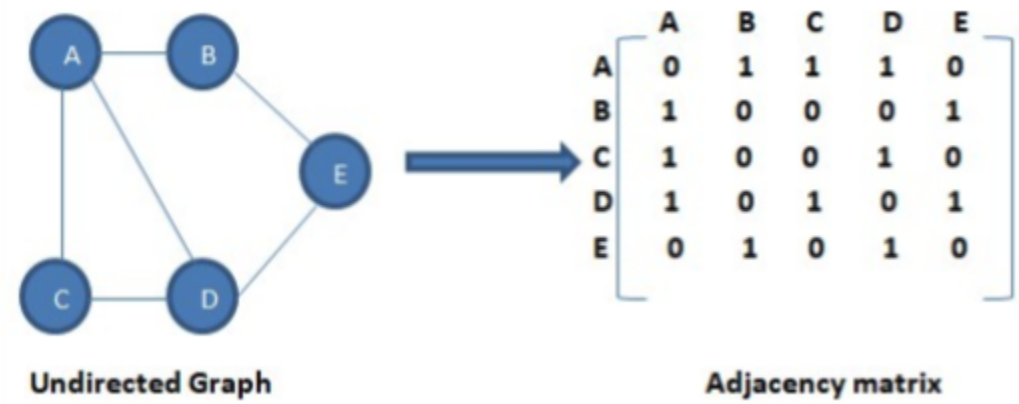
ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list

A.1. Create a Graph

Following are the basic operations that we can perform on the graph data structure:

- **Add a vertex:** Adds vertex to the graph.
- **Add an edge:** Adds an edge between the two vertices of a graph.
- **Display the graph vertices:** Display the vertices of a graph.

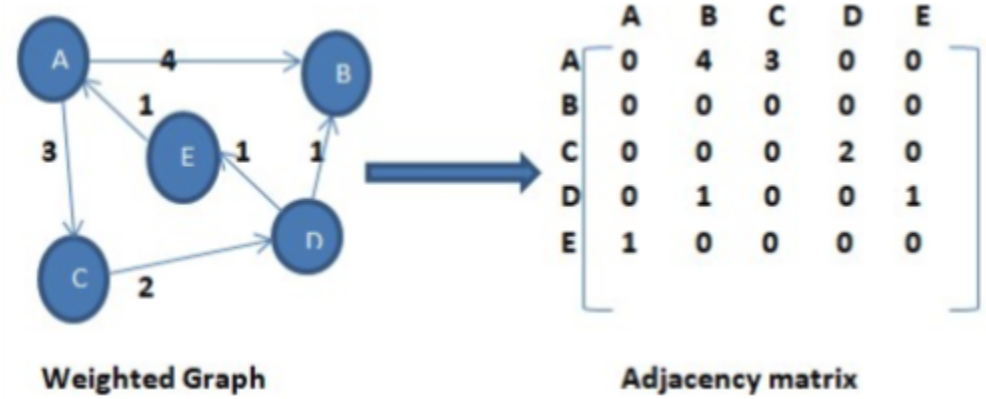
A.2. Adjacency Matrix



	A	B	C	D	E
A	0	1	1	1	0
B	1	0	0	0	1
C	1	0	0	1	0
D	1	0	1	0	1
E	0	1	0	1	0

Undirected Graph

Adjacency matrix

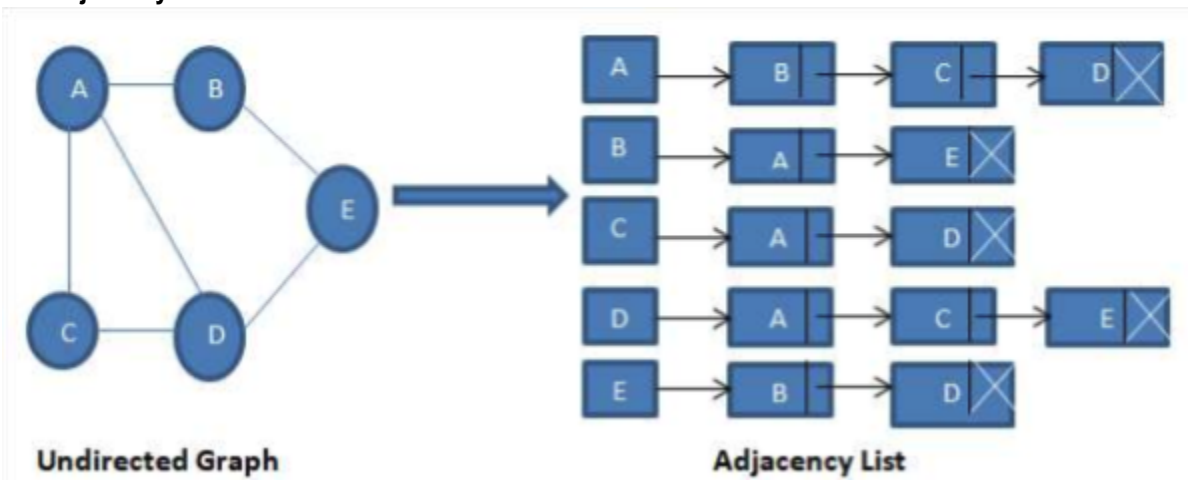


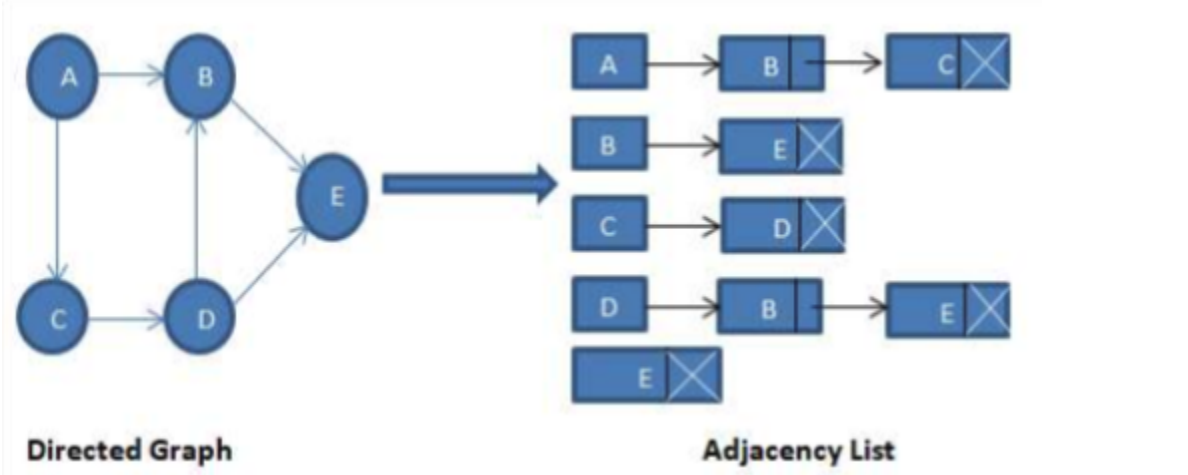
	A	B	C	D	E
A	0	4	3	0	0
B	0	0	0	0	0
C	0	0	0	2	0
D	0	1	0	0	1
E	1	0	0	0	0

Weighted Graph

Adjacency matrix

A.3. Adjacency List





Sample Code:

```
#include <iostream>

// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph{
```

```
private:
    // insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head; // point new node to current head
        return newNode;
    }
    int N; // number of nodes in the graph
public:
    adjNode **head; //adjacency list as array of pointers

    // Constructor
    DiaGraph(graphEdge edges[], int n, int N) {
        // allocate new node
        head = new adjNode*[N]();
        this->N = N;
        // initialize head pointer for all vertices
        for (int i = 0; i < N; ++i){
            head[i] = nullptr;
        }
        // construct directed graph by adding edges to it
        for (unsigned i = 0; i < n; i++) {
        int start_ver = edges[i].start_ver;
        int end_ver = edges[i].end_ver;
        int weight = edges[i].weight;
        // insert in the beginning
        adjNode* newNode = getAdjListNode(end_ver, weight,
head[start_ver]);
        // point head pointer to new node
        head[start_ver] = newNode;
        }
    }

    // Destructor
    ~DiaGraph() {
        for (int i = 0; i < N; i++)
```

```
        delete[] head[i];
        delete[] head;
    }

};
// print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i) {
    while (ptr != nullptr) {
    std::cout << "(" << i << ", " << ptr->val
    << ", " << ptr->cost << ") ";
    ptr = ptr->next;
    }
    std::cout << std::endl;
}

// graph implementation
int main() {
    // graph edges array.
    graphEdge edges[] = {
    // (x, y, w) -> edge from x to y with weight w
        {0,1,2},
        {0,2,4},
        {1,4,3},
        {2,3,2},
        {3,1,4},
        {4,3,3}
    };

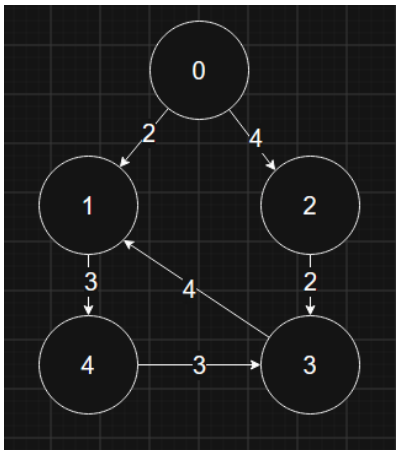
    int N = 6; // Number of vertices in the graph
    // calculate number of edges
    int n = sizeof(edges)/sizeof(edges[0]);

    // construct graph
    DiaGraph diagraph(edges, n, N);
    // print adjacency list representation of graph
    std::cout << "Graph adjacency list " << std::endl << "(start_vertex,
end_vertex, weight):" << std::endl;
```

```
    for (int i = 0; i < N; i++) {
        // display adjacent vertices of vertex i
        display_AdjList(diagraph.head[i], i);
    }

    return 0;
}
```

Vertices	Adjacent	Weight
0	2	4
0	1	2
1	4	3
2	3	2
3	1	4
4	3	3



$$\begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{ccccc} 0 & 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 2 & 4 & 0 & 0 \\ 0 & 0 & 0 & 0 & 3 \\ 0 & 0 & 0 & 2 & 0 \\ 0 & 4 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 \end{bmatrix} \end{array}$$

ILO B: Create C++ code for implementing graph traversal algorithms such as Breadth-First and Depth-First Search

B.1. Depth-First Search

Step 1. Include the required header files, as follows:

```
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>
```

```
template <typename T>
class Graph;
```

Step 2. Write the following struct in order to implement an edge in our graph:

```
template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;
    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const {
        return this->weight > e.weight;
    }
};
```

Step 3. Next, overload the << operator for the graph so that it can be printed out using the following function:

```
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G) {
    for (auto i = 1; i < G.vertices(); i++) {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges) {
            os << "{" << e.dest << ": " << e.weight << "}, ";
        }
        os << std::endl;
    }
    return os;
}
```

Step 4. Implement the graph data structure that uses an edge list representation as follows:

```
template <typename T>
class Graph {
public:
    // Initialize the graph with N vertices
    Graph(size_t N) : V(N){}
```

```
    // Return number of vertices in the graph
    auto vertices() const {
        return V;
    }

    // Return all edges in the graph
    auto &edges() const {
        return edge_list;
    }

    void add_edge(Edge<T> &&e) {
        // Check if the source and destination vertices are within range
        if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V) {
            edge_list.emplace_back(e);
        }else {
        std::cerr << "Vertex out of bounds" << std::endl;
        }
    }

    // Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list) {
            if (e.src == v){
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

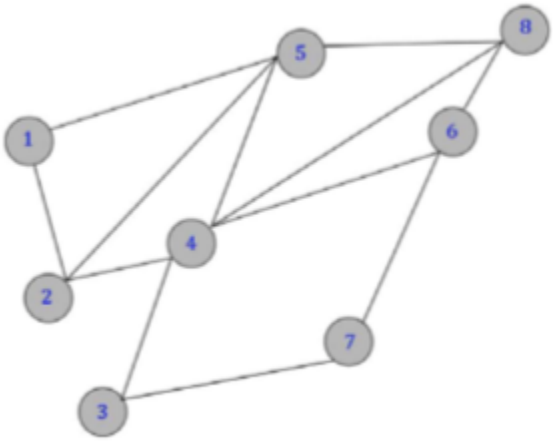
    // Overloads the << operator so a graph be written directly to a
stream
    // Can be used as std::cout << obj << std::endl;
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
private:
    size_t V; // Stores number of vertices in graph
```

```
    std::vector<Edge<T>> edge_list;
};
```

Step 5. Now, we need a function to perform DFS for our graph. Implement it as follows:

```
template <typename T>
auto depth_first_search(const Graph<T> &G, T dest) {
    std::stack<T> stack;
    std::vector<T> visit_order;
    std::set<T> visited;
    stack.push(1); // Assume that DFS always starts from vertex ID 1
    while (!stack.empty()) {
        auto current_vertex = stack.top();
        stack.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end()) {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex)) {
                // If the vertex hasn't been visited, insert it in the
stack.
                if(visited.find(e.dest) == visited.end()) {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}
```

Step 6. We shall test our implementation of the DFS on the graph shown here:



Use the following function to create and return the graph:

```
template <typename T>
auto create_reference_graph() {
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};
    for (auto &i : edges)
    for (auto &j : i.second)
    G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}
```

Note the use of null values for edge weights since DFS does not require edge weights. A simpler implementation of the graph could have omitted the edge weights entirely without affecting the behavior of our DFS algorithm.

Step 7. Finally, add the following test and driver code, which runs our DFS implementation and prints the Output:

```
template <typename T>
void test_DFS() {
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
```

```
    std::cout << G << std::endl;
    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
    std::cout << v << std::endl;
}

int main(){
    using T = unsigned;
    test_DFS<T>();
    return 0;
}
```

Step 8. Compile and run the preceding code. Include your output as a table in section 6.

```
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
```

B.2. Breadth-First Search

Step 1: Include the required header files and declare the graph as follows:

```
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
```

```
#include <queue>

template <typename T>
class Graph;
```

Step 2: Write the following struct, which represents an edge in our graph:

```
template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;
    inline bool operator<(const Edge<T> &e) const {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const{
        return this->weight > e.weight;
    }
};
```

Step 3: Next, overload the << operator for the Graph data type in order to display the contents of the graph:

```
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G){
    for (auto i = 1; i < G.vertices(); i++){
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges) {
            os << "{" << e.dest << ": " << e.weight << "}, ";
        }
        os << std::endl;
    }
}
```

Step 4: Write a class to define our graph data structure, as shown here:

```
template <typename T>
class Graph {
public:
    Graph(size_t N) : V(N) {}
    auto vertices() const{
        return V;
```

```
    }
    auto &edges() const{
        return edge_list;
    }
    void add_edge(Edge<T> &&e){
        if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V){
            edge_list.emplace_back(e);
        }
        else {
            std::cerr << "Vertex out of bounds" << std::endl;
        }
    }
    auto outgoing_edges(size_t v) const {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list) {
            if (e.src == v) {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }
    template <typename T>
    friend std::ostream &operator<<(std::ostream &os, const Graph<T> &G);
private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};
```

Step 5: For this exercise, we shall test our implementation of BFS on the following graph:
We need a function to create and return the required graph. Note that while edge weights are assigned to each edge in the graph, this is not necessary since the BFS algorithm does not need to use edge weights. Implement the function as follows:

```
template <typename T>
auto create_reference_graph() {
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
```

```
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};
    for (auto &i : edges)
    for (auto &j : i.second)
    G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}
```

Step 6: Implement the breadth-first search like so:

```
template <typename T>
auto breadth_first_search(const Graph<T> &G, int dest) {
    std::queue<int> queue;
    std::vector<int> visit_order;
    std::set<int> visited;
    queue.push(1); // Assume that BFS always starts from vertex ID 1
    while (!queue.empty()) {
        auto current_vertex = queue.front();
        queue.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end()) {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
            queue.push(e.dest);
        }
    }
    return visit_order;
}
```

Step 7: Add the following test and driver code that creates the reference graph, runs BFS starting from vertex 1, and outputs the results:

```
template <typename T>
void test_BFS() {
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
```

```
    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
    std::cout << v << std::endl;
}

int main() {
    using T = unsigned;
    test_BFS<T>();
    return 0;
}
```

Include the out and observation as a table in section 6.

```
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

BFS Order of vertices:
1
2
5
4
8
3
6
7
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

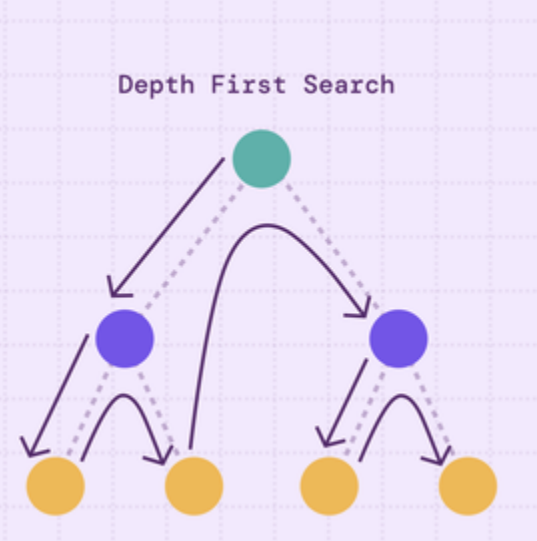
1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore another vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.

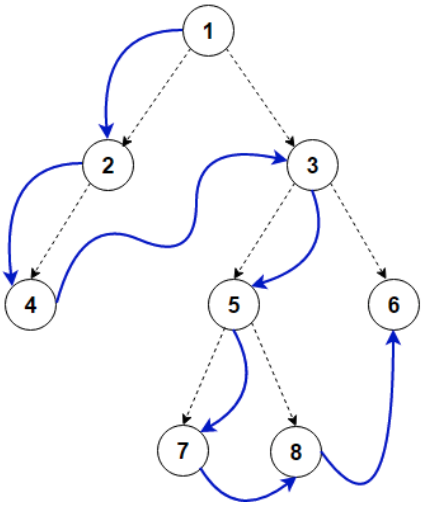
The most helpful algorithm to complete this task is depth first search because it clears one vertex until it clears all its contents before moving on to another, which is exactly what their goal is. The implementation will utilize a stack data structure.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

In tree traversal, DFS is equivalent to pre-order traversal, it goes through one branch/vertex until the leaf

node, then moves on to another branch/vertex after clearing it.
Graphical Comparison:





3. In the performed code, what data structure is used to implement the Breadth First Search?
In performing a breadth first search, the program utilizes a queue data structure because it is a FIFO system. When looking at a vertex, it first lists all the adjacent into a queue, then dequeues them one by one checking each vertex's adjacent vertices and queueing it into the list, making it travel through all the vertices in one level before moving on to the next level.

4. How many times can a node be visited in the BFS?
In breadth first search, a node can be visited a maximum amount of one time because the algorithm checks if the node has been visited and skips it if it has. Similarly, DFS also does this with its traversal, only having difference in the implementation method; BFS using queue and DFS using stacks.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

COMPUTER ENGINEERING

QUEZON CITY

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.